

APLICACION: RETEVE

JOSHUA FABIÁN GUERRA CASTILLO

Tecnológico de Costa Rica

RETEVE

Proyecto #3

Taller de programación

William Mata

18-06-2023

1. Enunciado del proyecto
2. Temas investigados
3. Diseño de la solución
4. Conclusiones
5. Estadística de tiempos
6. Lista de revisión

Enunciado:

Desarrollar un programa para administrar una parte de un proceso de revisión técnica de vehículos llevada a cabo en una estación especializada. La revisión consiste en examinar vehículos para obtener un resultado de la revisión y si el vehículo es apto para circular se emite un certificado de tránsito. De no ser apto para circular este certificado no se emite, en su lugar el vehículo necesitará reinspección o podría ser que la revisión determine que el vehículo debe sacarse de circulación. El programa tendrá funciones para el registro de las citas de revisión de los vehículos, el ingreso de vehículos a la estación, un tablero que muestra en qué parte de la revisión está el vehículo dentro de la estación, el obtener los resultados de la revisión, una lista de posibles fallas y datos de configuración del sistema.

Para efectos de esta especificación, cuando se considere necesario se hará referencia a los valores mostrados actualmente en la sección de configuración, por ejemplo, cuando se hable de cantidad de líneas de trabajo en la estación será 6 que es su valor actual.

Tenemos entonces que en la estación hay 6 líneas de trabajo: una línea de trabajo es una cola (primero que entra, primero que sale) donde están los vehículos a revisar.

En cada línea de trabajo el vehículo debe pasar secuencialmente por 5 puestos de revisión:

- Puesto 1: revisiones generales (luces, llantas, vidrios, etc.)
- Puesto 2: banco de amortiguadores

- Puesto 3: sistema de frenado
- Puesto 4: detector de holguras
- Puesto 5: emisión de contaminantes

En cada puesto puede haber solo un vehículo, y cuando sale del puesto puede estar sin fallas o con fallas leves o graves.

Al final de la línea de trabajo, después del puesto 5, un vehículo aprueba o no aprueba la revisión técnica.

“Cola de espera”: hay una por cada línea de trabajo, antes del puesto 1 los vehículos están en esta cola esperando la entrada a ese puesto de revisión.

“Cola de revisión”: hay una por cada línea de trabajo, cuando los vehículos entran a la revisión, a partir del puesto 1, pasan de la “cola de espera” a la “cola de revisión”. Esto causa una salida de la “cola de espera” y una entrada en la “cola de revisión”.

Toda la información del sistema debe ser persistente, es decir, se debe preservar la información actualizada de tal manera que pueda estar disponible en cada corrida del programa, para ello puede usar archivos.

El programa tendrá un menú principal implementado con botones (widget Button) desde el cual se accederán las funciones del programa. Cada opción debe ofrecer la posibilidad de regresar al menú anterior preservando la información registrada. Usted define la interfaz gráfica siempre y cuando cumpla con los requerimientos funcionales del programa indicados seguidamente.

Temas investigados:

Módulos:

Tkinter: Tkinter es un módulo para crear una interfaz gráfica (GUI) el cual está incluido en la biblioteca estándar de Python, es muy simple y fácil de usar ya que tiene una sintaxis clara y concisa, tiene diversas funciones, entre ellas varios widgets, algunos son:

-Button() el cual sirve para crear botones.

-variable = Tk() que sirve para crear ventanas.

-ventana.withdraw() que sirve para cerrar una ventana.

-Label() sirve para crear etiquetas en las que se pueden ingresar datos.

-Entry() sirve para ingresar datos en una ventana

Para efectos de este proyecto se importó el módulo entero usando “from tkinter import*”, esto me permite usar todo el módulo, además se importó “messagebox” para mostrar mensajes de error o de información al usuario, se muestran los mensajes con “showwarning”, “showerror” y “showinfo”.

Json: Viene instalado automáticamente en Python, incluye funciones para trabajar con archivos y cadenas de caracteres JSON.

Para efectos de este proyecto se utilizó para el manejo de datos en archivos con json.loads()

Smtplib: Es un objeto o sesión de cliente SMTP que se puede usar para enviar correos.

Para este proyecto se utilizó `smtplib.SMTP("smtp.gmail.com", #servidor)`

Decouple: Sirve para organizar ajustes para así poder cambiar parámetros sin tener que volver a cargar una app, creada por Django

Para este proyecto se usó "config" para la contraseña del correo

Datetime: Proporciona clases para manipular fechas y horas y permite realizar operaciones aritméticas con estos datos

Para este proyecto se utilizó `datetime.datetime.now()` para conseguir la fecha actual y los comandos `.year`, `.month`, `.day` y `.strftime("%I")` para conseguir el año, mes, día y hora en formato hh:mm

Validate_email: Sirve para verificar que un email sea válido, es decir, que su sintaxis sea correcta

Para este proyecto solo se utilizó `validate_email`, para verificar que fuera un correo válido

Email.mime: Puede crear una nueva estructura de objeto mediante la creación de una instancia de `Message`, añadiendo accesorios, para este proyecto se utilizó `.multipart`, `.base` y `.text`

Encoders: Son codificadores para mensajes, en este proyecto se utilizó el método `encode_base64()`.

Queue: Es un módulo que implementa colas multi-productor y multi-consumidor, sirve muy bien para programación en hilo, cuando la información debe intercambiarse de forma segura entre subprocesos. Para este proyecto se usó Queue para crear las colas y los comandos `.get()` y `.put()` para manejarlas. También se usó `.empty()` para recorrerlas.

Os: Este módulo también viene instalado en la biblioteca de Python, y brinda funciones para interactuar con archivos del sistema operativo tales como:

- os.remove(dirección)** la cual sirve para eliminar un archivo.
- os.mkdir(dirección)** sirve para crear un nuevo directorio.
- os.rename(actual, nuevo)** recibe como primer parámetro la dirección actual y como segunda la dirección nueva, esto para cambiar el nombre del archivo.

Para este proyecto se utilizó solo la función **os.system(comando)**, como parámetro recibió la dirección del manual de usuario para de esta forma desplegarlo al usuario cada vez que presione el botón “Ayuda”.

Arboles:

Para poder llevar el control de las citas en un ABB se usaron clases como

- **Class Nodo:** La cual se encargaba de crear los nodos para las citas

- **Class Arbol:** La cual se encargaba de hacer todas las acciones sobre las citas utilizando recursión de pila, usa funciones como:
- Agregar, agregar_recursivo, buscar, buscar_recursivo y otras, cada una hace una función para ser usadas en distintas partes del programa, son usadas como métodos y de la clase árbol, como por ejemplo “citas.agregar(cita_actual)”, con este comando se llama a la función “agregar” la cual llama a la función “agregar_recursivo” la cual agrega la cita a un nodo del árbol llamando a la clase Nodo y pasándole los datos para que le busque un nodo vacío.

Diseño de la solución:

Árbol de citas:

Para manejar el árbol de citas se usaron clases, la primera fue Nodo:

```
#Clase para los nodos del arbol
class Nodo:

    #__init__ es un nombre especial
    #E: Una cia
    #S: Acomoda las citas en el arbol
    EasyCode: Explain
    def __init__(self, datos):
        self.datos = datos
        self.izquierda = None
        self.derecha = None

    #__str__ es un nombre especial
    #E: Nada
    #S: Convierte los datos a str
    EasyCode: Explain
    def __str__(self):
        return str(self.datos)

    EasyCode: Explain
    def get_cita(self):
        return self.datos[0]

    EasyCode: Explain
    def get_placa(self):
        return self.datos[8]
```

En esta clase se usaron 4 funciones:

`__init__` es un nombre especial para crear los datos a los nodos, recibe los datos a agregar y les busca un lugar en el árbol.

La función `__str__` también tiene un nombre especial, esta clase sirve para trabajar el árbol con strings.

La función `get_cita` sirve para cuando se deba hacer una búsqueda en el árbol y la función `get_placa` sirve para encontrar la placa en el árbol, cuando se necesite.

También se usó una clase llamada Arbol:

```
class Arbol:

    #__init__ es un nombre especial
    #E: Una cia
    #S: Llama a la clase Nodo para que acomode las citas en el arbol
    EasyCode: Explain
    def __init__(self, datos):
        self.raiz = Nodo(datos)

    #Agregar de forma recursiva
    #E: Un nodo del arbol y los datos a agregar
    #S: Actualiza el arbol con una nueva cita
    EasyCode: Explain
    def agregar_recursivo(self, nodo, datos):
        if datos[0] < nodo.datos[0]:
            if nodo.izquierda is None:
                nodo.izquierda = Nodo(datos)
            else:
                self.agregar_recursivo(nodo.izquierda, datos)
        else:
            if nodo.derecha is None:
                nodo.derecha = Nodo(datos)
            else:
                self.agregar_recursivo(nodo.derecha, datos)

    #Funcion publica
```

```
#E: Cita
#S: Llama a la funcion recursiva del arbol
EasyCode: Explain
def agregar(self, datos):
    self.agregar_recursivo(self.raiz, datos)

#Eliminar de forma recursiva
#E: Un nodo del arbol y los datos a eliminar
#S: Actualiza el arbol con una cita menos
EasyCode: Explain
def eliminar_recursivo(self, nodo, datos):
    if nodo is None:
        return nodo

    if datos[0] < nodo.datos[0]:
        nodo.izquierda = self.eliminar_recursivo(nodo.izquierda, datos)
    elif datos[0] > nodo.datos[0]:
        nodo.derecha = self.eliminar_recursivo(nodo.derecha, datos)
    else:
        if nodo.izquierda is None:
            temp = nodo.derecha
            nodo = None
        else:
            temp = nodo.izquierda
            nodo = None
        return temp
```

```
#S: Llama a la funcion recursiva del arbol
EasyCode: Explain
def eliminar(self, datos):
    self.raiz = self.eliminar_recursivo(self.raiz, datos)

    EasyCode: Explain
    def encontrar_minimo(self, nodo):
        actual = nodo
        while actual.izquierda is not None:
            actual = actual.izquierda
        return actual

    #Buscar de forma recursiva
    #E: Un nodo del arbol y los datos a buscar, en este caso el numero de cita y la placa
    #S: Devuelve la cita que se busca en caso de que exista
    EasyCode: Explain
    def buscar_recursivo(self, nodo, dato1, dato2):
        if nodo is None or nodo.datos[0] == dato1 and nodo.datos[8] == dato2:
            return nodo

        if dato1 < nodo.datos[0]:
            return self.buscar_recursivo(nodo.izquierda, dato1, dato2)
        else:
            return self.buscar_recursivo(nodo.derecha, dato1, dato2)
```

```
EasyCode: Explain
def buscar(self, dato1, dato2):
    return self.buscar_recursivo(self.raiz, dato1, dato2)

#Buscar de forma recursiva, es distinta a la anterior para buscar otros dos datos especificos
#E: Un nodo del arbol y los datos a buscar, en este caso el mes y la placa
#S: Devuelve la cita que se busca en caso de que exista
EasyCode: Explain
def buscar_recursivo2(self, nodo, dato1, dato2):
    if nodo is None or nodo.datos[3] == dato1 and nodo.datos[8] == dato2:
        return nodo

    if dato1 < nodo.datos[3]:
        return self.buscar_recursivo2(nodo.izquierda, dato1, dato2)
    else:
        return self.buscar_recursivo2(nodo.derecha, dato1, dato2)

#Funcion publica
#E: Cita
#S: Llama a la funcion recursiva del arbol
EasyCode: Explain
def buscar2(self, dato1, dato2):
    return self.buscar_recursivo2(self.raiz, dato1, dato2)
```

```

lenovo > Desktop > retele.py > opcion
def reemplazar_datos_recursivo(self, nodo, dato_viejo, dato_nuevo):
    if nodo is None:
        return None

    if nodo.get_cita() == dato_viejo.get_cita() and nodo.get_placa() == dato_viejo.get_placa():
        nodo.datos = dato_nuevo

    elif dato_viejo.get_cita() < nodo.get_cita():
        nodo.izquierda = self.reemplazar_datos_recursivo(nodo.izquierda, dato_viejo, dato_nuevo)

    else:
        nodo.derecha = self.reemplazar_datos_recursivo(nodo.derecha, dato_viejo, dato_nuevo)

    return nodo

#funcion publica
#E: Cita viejo, cita actualizada
#llama a la funcion recursiva
#ayuda: Eplain
def reemplazar(self, dato_viejo, dato_nuevo):
    self.raiz = self.reemplazar_datos_recursivo(self.raiz, dato_viejo, dato_nuevo)

```

Este árbol tiene una función para inicializarlo “__init__” y sus otras funciones son las que hacen las operaciones recursivas dentro del mismo árbol, como agregar que llama a agregar_recursivo, el cual le busca un campo a la nueva cita junto con la clase nodo, para crearle un nodo nuevo a cada cita agregada, también tiene funciones para eliminar, reemplazar y buscar en el árbol, utiliza la palabra self. Para que el programa sepa que debe trabajar sobre la misma clase.

La función “agregar recursivo” se relaciona con la clase Nodo ya que dicha clase crea un espacio nuevo y agregar se encarga de verificar en qué dirección lo va a agregar.

Archivos:

Se utilizó solamente un archivo llamado “configuración.dat”, se usó para ingresar al programa la configuración inicial y también para que pudiera ser modificado mediante la opción “configuración del sistema”, también

se utilizó para validaciones como las de las horas de abrir y cerrar de la instalación.

```
#Datos de configuracion iniciales
#[líneas, [horas], min, días para reinspection, fallas, meses, IVA, [tarifas]]
dat = [6, [1, 24], 20, 30, 4, 5, 13.0, [10920, 14380, 14380, 11785, 14380, 7195, 14380, 6625]]

#Inicializacion de archivo y guardado de datos
configdat=open("configuracion.dat", "w")
configdat.write(str(dat))
configdat.close()
```

El archivo contiene una lista con las líneas de trabajo, una sublista con la hora de inicio y hora de fin de la instalación, los minutos por cada revisión, los días para sacar una reinspection, cuantas fallas es el máximo, los meses que estará abierto el local, el impuesto IVA y una sublista con las tarifas para cada tipo de vehículo.

Para ingresar a los datos del archivo se usaron dos formas:

La forma de usar el método “open()” y el modo de abrirlo, en este caso “r” ya que se iba a asignar los datos del archivo a una variable para las validaciones, esto se hizo con el método .read y eval()

```
configdat = open("configuracion.dat", "r")
datos = configdat.read()
elementos = eval(datos)
configdat.close()
```

La otra manera usada fue el método “With open() as archivo”

Luego se agregaron los datos a una variable con el método .loads() del módulo Json.

```
with open("configuracion.dat", "r") as archivo:
    datos = archivo.read()
    elementos = json.loads(datos)
archivo.close()
```

Se usaron variables StringVar() para guardar los datos de las listbox:

```
op = StringVar()
```

Se crearon dos funciones para el envío de correos: La primera enviaba la notificación de que la cita había sido creada, recibía el correo y una lista con la fecha y hora y utilizaba el módulo smtplib y el módulo decouple. Smtplib para el envío y Decouple conseguía la contraseña de la variable de entorno.

La otra función servía para enviar el PDF, hacia una validación en caso de no haber conseguido el certificado de transito, si no se conseguía no se enviaba:

```
def env_correo(receptor, fyh):
    mensaje = f"""Subject: Su cita ha sido programada!

    Cita programada para el día {fyh[0]}/{fyh[1]}/{fyh[2]} a las {fyh[3]}:{fyh[4]}.
    Gracias por preferirnos!

    Ríeteve...
    """.encode('utf-8')

    serv = smtplib.SMTP("smtp.gmail.com", 587)
    serv.starttls()
    serv.login("pruebap3jfgc@gmail.com", config("contra"))
    serv.sendmail("pruebap3jfgc@gmail.com", receptor, mensaje)
    serv.quit()
```

```
> Lenovo > Desktop > retrieve.py > env_res
def env_res(receptor, val_co):
    remite = "pruebap3jfgc@gmail.com"
    contraseña = config("contra")
    os[0] + ".jpg")
    mensaje = MIMEBase('application', 'octet-stream')
    mensaje['From'] = remite
    mensaje['To'] = receptor
    mensaje['Subject'] = "Resultado de revisión"

    if val_co:
        pdfs = ["C:/Users/Lenovo/Desktop/Certificado.pdf", "C:/Users/Lenovo/Desktop/Resultado de la revision.pdf"]
    else:
        pdfs = ["C:/Users/Lenovo/Desktop/Resultado de la revision.pdf"]

    for pdf in pdfs:
        adjunto = MIMEBase('application', 'octet-stream')
        adjunto.set_payload(open(pdf, 'rb').read())
        encoders.encode_base64(adjunto)
        adjunto.add_header('Content-Disposition', f'attachment; filename="{pdf}"')
        mensaje.attach(adjunto)

    contenido_mensaje = "Subject: Resultado de su cita\n\n"
    contenido_texto = MIMEText(contenido_mensaje, 'plain')
    mensaje.attach(contenido_texto)

    servidor_smtp = smtplib.SMTP('smtp.gmail.com', 587)
    servidor_smtp.starttls()
    servidor_smtp.login(remite, contraseña)
    servidor_smtp.sendmail(remite, receptor, mensaje.as_string())
    servidor_smtp.quit()
```

```
ls / Lenovo / Desktop /  Retrive.py  Arbol

#Modulos
#Interfaz
from tkinter import *
from tkinter import messagebox
#Guardar en archivos
import json
#Correos
import smtplib
from decouple import config
from validate_email import validate_email
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email.mime.text import MIMEText
from email import encoders

#Horarios y citas
import datetime

#Colas
from queue import Queue

#Manual de usuario
import os

#PDF
from reportlab.pdfgen import canvas
```

Tkinter: Creación de la interfaz

Messagebox: Para el despliegue de mensajes de error, información y advertencia.

Json: Para manejar los datos de los archivos.

Smtplib: Envío de correos a un correo existente

Decouple: Acceder a la contraseña que estaba en una variable de entorno.

Validate_email: Valida la sintaxis de los correos

Email.mime: Se usaron multipart, base y text para convertir los PDF a un dato válido para su envío.

Encoders: Para en el de los datos .mime.

Datetime: Para tener un manejo en tiempo real de las horas

Queue: Para la creación y manejo de las colas.

Os: Despliegue del manual de usuario.

Conclusiones:

Problemas:

Tuve problemas con los árboles, ya que no los estaba haciendo con clases, los estaba haciendo como se vio en clase, con listas, pero luego leí las instrucciones y decía que buscara otra manera de implementarlos, entonces encontré que se podía con clases y tuve que adaptar las clases a mi aplicación, tarde aproximadamente 6 horas en eso, pero al final funcionó.

Correo: Ya que en el primer proyecto no había hecho lo del correo no tenía un correo para hacer esas pruebas y tuve que crear uno, luego tuve que buscar la manera de permitirle a Python ingresar al correo, ya que Google deshabilitó la opción para permitir acceso a aplicaciones poco seguras, la solución fue solicitar una contraseña a Google, crear una variable de entorno en mi laptop y usando la función **“config”** del módulo **“decouple”** llamar a esa variable, luego tuve problemas con la información que envía el correo ya que me daba errores por datos que no se podían decodificar, la solución fue estructurar el mensaje de otra manera y usar el comando **“mensaje.encode(‘utf-8’)”** el cual me permitió decodificar todos los datos del mensaje y enviarlos al destinatario.

Receptor: Tuve problemas con el correo receptor, ya que ese string se guardaba en el Entry “correo” y aunque al final de la función para

guardar los datos en una lista que se envia a la clase árbol ára guardarla tenía un llamado a la función “env_correo(correo, fyh)” pero a la hora de enviar el correo daba error por que el receptor no era válido, probé con diferentes correos pero al final el error era porque antes de enviar el Entry correo a la función le eliminaba los datos a todos los Entry de cita, lo que significa que cada vez que enviaba correo a la función env_correo recibía un correo con los datos “” en él, lo que hacía que diera error, la solución fue que antes de borrar los datos le asignara a la variable “receptor” los datos de “correo”, luego pasarle esa variable a env_correo y funcionó bien.

Enviar PDF: No sabia como hacerlo y tuve que importar 4 librerias más, con esto tuve más problemas porque no sabia como usar esas librerías, los solucioné investigando

Tuve problemas con muchas de las validaciones, ya que eran muy especificas y me llegué a enredar, la solución fue dedicarles unos minutos de análisis y después de unos minutos encontré los errores de lógica y listo.

Tablero de revisión: Tuve muchos problemas ya que no encontraba la manera de refrescar el tablero y solo pude reflejar un cambio, también tuve problemas con los comandos, los cuales eran problemas de lógica, los arreglé después de dedicarles tiempo.

Aprendizajes:

Mientras desarrollaba este proyecto pude aprender a usar un ABB gracias a lo que investigue, también aprendí a usar clases en mi código y descubrí que es muy fácil y agiliza más al programa, aprendí a recorrer una cola queue y cómo usarla, esto antes no lo tenía muy claro, aprendí a enviar correos desde Python ya que en el primer proyecto no hice esa parte, me arrepiento porque era muy simple, también aprendí la importancia de las variables globales, también profundice más en la recolección de datos a partir de un Entry, con el comando **.get()**, ya que en proyectos anteriores no era tan necesario como en este, otra cosa importante que aprendí es como crear ventanas con una Listbox, también como crear opciones múltiples para escoger.

Aprendí a enviar un PDF por correo y al volver a crear un PDF en Python reforcé esos conocimientos.

Actividad realizada	Horas
Análisis del problema	3
Diseño de algoritmos	4
Investigación de Árboles y demás elementos del programa	3
Programación	30
Documentación interna	2
Pruebas	6
Elaboración del manual de usuario	1
Elaboración de documentación del proyecto	1
TOTAL	50 horas

Concept	Puntos	Puntos obtenidos	Avance 100/%/0	Análisis de resultados
Programación de citas implementando ABB con recursión	25	25	100	
Asignación manual de citas	2	2	100	
Asignación automática de citas	5	5	100	
Cancelar citas	2	2	100	
Ingreso de vehículos a la estación	5	5	100	
Despliegue del tablero de revisión	10	10	100	
Ejecución de los comandos del tablero	12	5	40	Se ejecutan 3 de los 5 comandos
Registro de fallas por cita de revisión	5	5	100	
Resultado de la revisión en PDF	5	5	100	
Certificado de transito	3	3	100	
Envío de correos electrónicos (cita, resultados de la revisión)	5	5	100	
Lista de fallas (CRUD)	10	10	100	
Configuración del sistema	6	6	100	
Ayuda: manual de usuario en pantalla	5	5	100	
TOTAL	100	93		
Funcionalidades adicionales				

Bibliografía:

Cabrera, L. (2021). Árbol binario en Python - Estructura de datos en programación. [Video]. Recuperado de https://www.youtube.com/watch?v=d0ibZK_6Q7g

Cassingena Estefania. (2022). Python leer archivo JSON – Cómo cargar JSON desde un archivo y procesar dumps. Recuperado de <https://www.freecodecamp.org/espanol/news/python-leer-archivo-json-como-cargar-json-desde-un-archivo-y-procesar-dumps/#:~:text=JSON%20es%20un%20formato%20de,ejecuta%20el%20programa%20de%20Python.>

Conde, J. (2013). GitHub. Crear un Completo sistema de control de versiones en minutos. [Video]. Recuperado de <https://www.youtube.com/watch?v=W6yQQPVhR80>

CosmicLearn. (s. f.). MÓDULO DE PYTHON OS. Recuperado de <https://www.cosmiclearn.com/lang-es/python-os.php#:~:text=El%20m%C3%B3dulo%20os%20en%20Python,a%20otra%20del%20siguiente%20comando.>

Python Software Foundation. (2023). datetime — Tipos básicos de fecha y hora. Recuperado de <https://docs.python.org/es/3/library/datetime.html#:~:text=El%20m%C>

3%B3dulo%20datetime%20proporciona%20clases,su%20posterior%
20manipulaci%C3%B3n%20o%20formateo.

python-decouple 3.8. (2023). Recuperado de <https://pypi.org/project/python-decouple/>

Python Software Foundation. (2023). queue — clase de cola sincronizada.

Recuperado de

<https://docs.python.org/es/3.8/library/queue.html#:~:text=El%20m%C3%B3dulo%20queue%20implementa%20colas,la%20sem%C3%A1ntica%20de%20bloqueo%20necesaria.>

Redacción KeepCoding. (2023). ¿Qué es Tkinter? Recuperado de

[https://keepcoding.io/blog/que-es-](https://keepcoding.io/blog/que-es-tkinter/#:~:text=implementar%20lo%20aprendido%3F-)

[tkinter/#:~:text=implementar%20lo%20aprendido%3F-](https://keepcoding.io/blog/que-es-tkinter/#:~:text=implementar%20lo%20aprendido%3F-)

,Qu%C3%A9%20es%20Tkinter%3A%20una%20librer%C3%ADa%20de%20Python,gr%C3%A1fica%20de%20escritorio%20con%20Python.

Vaati Esther. (2022). Enviar correos electrónicos en Python con SMTP.

Recuperado de <https://code.tutsplus.com/es/tutorials/sending-emails-in-python-with-smtp--cms-29975>

validate_email 1.3. (2015). Recuperado de

https://pypi.org/project/validate_email/

email.encoders. (2023). In Python Software Foundation. Retrieved June 18,

2023, from <https://docs.python.org/es/3/library/email.encoders.html>

email.mime: Creating Email and MIME Objects from Scratch. (2023). In

Python Software Foundation. Retrieved June 18, 2023, from

<https://docs.python.org/es/3.8/library/email.mime.html>